Noveris

Learn Together. Grow Further. — a GenAI peer-learning companion

1. Project objective

Peer learning works when people explain, question and correct each other, but real study groups are unbalanced: one voice dominates, others hesitate to ask, wrong explanations go unchallenged, and a single explanation style does not fit everyone. **Noveris** is a working end-to-end learning companion that turns a course document into guided individual practice and structured peer study. The AI acts as **tutor** (hints before answers, level- and style-adapted explanations), **facilitator** (peer-first prompts in a shared room) and **planner** (next step), with revision-pinned retrieval and citation-provenance checks for course tutoring. These checks do not establish the correctness of every generated claim. Protected course and room operations use deterministic server access checks.

2. What Noveris does — mapped to the challenge goals

Challenge goal	Implemented in Noveris
Smart peer learning support	**Study Together** rooms (six participants by default; server configuration allows 2–10; join by code, host/presenter roles, shared activity, private checks then group debrief). A **peer-first facilitator** proposes a question or hint that invites a peer explanation before any answer is revealed. The core text flow separates peer chat from explicit facilitator actions; phase checks and tentative confusion signals can also trigger local guidance. Optional voice-moderator scaffolding is separate.
Explain simply, guiding questions, hints before answers, confusion detection, next step	**Nardouch** course tutor with modes (explain, Socratic, peer) and one-tap “Explain simply / Another example / Step by step / Go deeper”. Practice support enforces a **graduated hint before the full explanation**. Wrong attempts and hint escalation raise tentative **confusion signals**. Progress and a planner suggest the next step.
Diagnostic + personalisation	A 6-question self-report **learning-style probe** (visual, analogy, step-by-step, example, repetition, problem-solving; each style offered equally) selects how explanations are phrased. It is explicitly provisional: it never grades and is visible only to the learner.
Multilingual (Arabic, Darija, French, English)	Nardouch text language policy targets Darija in *Arabizi* or Arabic script, French, English and code-switching, with a terminology glossary and a Tunisian-Darija evaluation set (`apps/ai/agents/nardouch/evaluation/cases.json`). Course content may itself be French (see Fig. 1).
Open innovation	**Visual Learning Lab** (model proposes a scenario, trusted code simulates it), versioned **Course Studio** (upload → OCR/extraction → instructor review → publish), instructor insights dashboard, XP/badges and progress, optional voice input and answer playback.
Entrepreneurship angle	Hybrid institutional + independent model (§12).

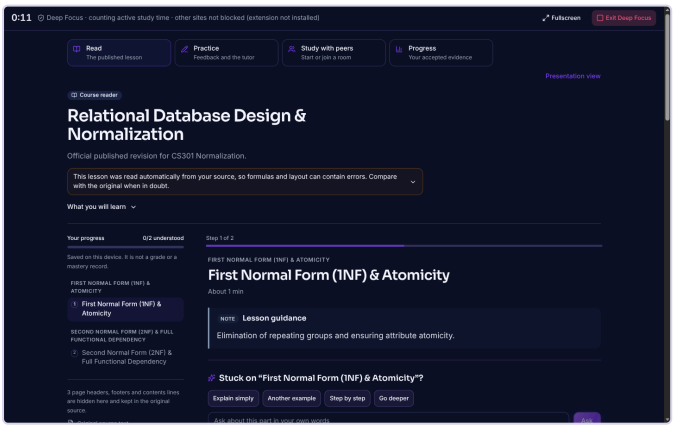


Fig. 1. Course reader (seeded demo course): guided tutor shortcuts, progress, source-grounded lesson.

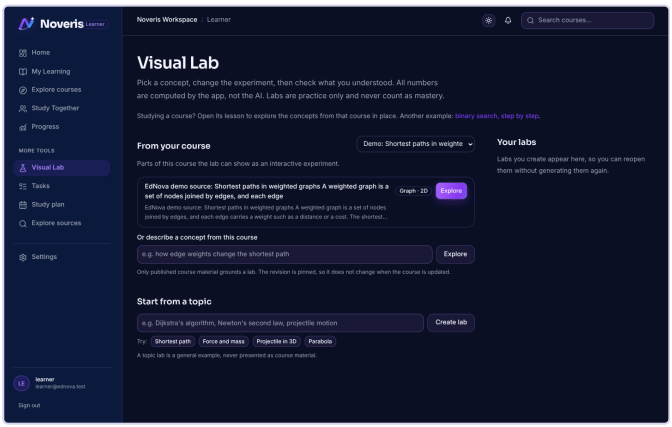


Fig. 2. Visual Lab: AI proposes, the app computes; labs never count as mastery.

Screenshots supplied with the original report show the seeded demo. They were retained for this source review; no fresh application capture is claimed.

3. System architecture

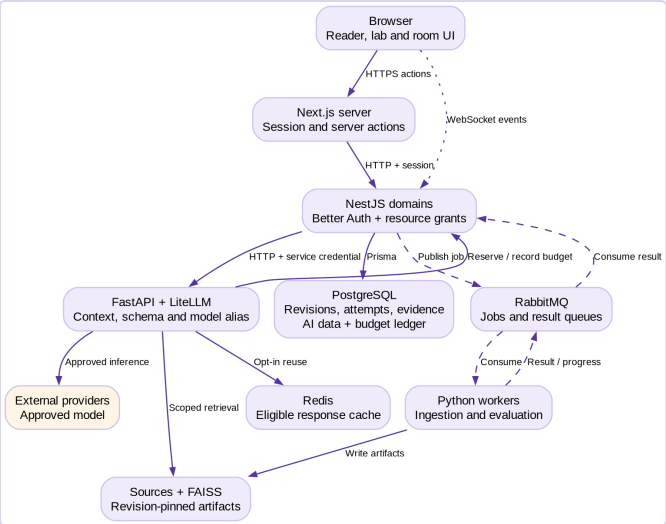


Fig. 3. Runtime architecture. Solid: synchronous HTTP; dashed: broker jobs; dotted: WebSocket.

Layer	Technology and responsibility
Web	Next.js 16 / React 19 / Tailwind; server actions; Better Auth sessions; WCAG-minded semantics (skip link, labelled controls, keyboard focus, RTL-capable text).
API (control plane)	NestJS domains on Node 24, Prisma 7 + **PostgreSQL**. Owns identity, course/room access grants, published revisions, attempts, evidence and the **inference budget ledger**.
AI service	Python FastAPI + LiteLLM. Agents: tutor, coach, planner, evaluator, facilitator, Visual Lab selector/planner, course ingestion. Schema-validated outputs.
Workers	RabbitMQ jobs with progress events: ingestion (text, OCR, chunking, embeddings), evaluation, planning.
Retrieval	all-MiniLM-L6-v2 (384-d, normalised) + FAISS `IndexFlatIP`, 200-word chunks with 50-word overlap, **pinned to a course revision**.
Cache / realtime	Redis for opt-in response reuse and rate state; WebSocket for room state, presence and facilitator events. Optional voice uses separate adapters.

Core design decisions

- **Models never hold authority.** NestJS decides who can read, join, submit and publish; the model only produces text or a structured proposal that the API validates (shape, recipient, room sequence, citation provenance) before delivery.
- **Versioned sources.** A `CourseRevision` records the source manifest and index hashes. Activities and rooms pin a reviewed revision, so retrieval and citations stay stable when a draft changes.
- **Two access models, one engine.** One identity can hold institution memberships (instructor assignment, student enrolment) and personal courses. Membership, enrolment, source grant and room membership are separate checks; a join code never grants document access.
- **Idempotent learning evidence.** A submission is bound to learner, attempt ID, activity revision and payload hash; replays reuse the stored receipt, conflicting payloads are rejected, and the server (not the browser) evaluates against stored criteria.

Typical flow

Instructor uploads a PDF → worker extracts/OCRs and chunks it → AI proposes objectives and activities → instructor reviews and publishes a revision → learners read, request graduated hints and ask the grounded tutor → a peer opens a room pinned to that revision → each member submits a *private* check → the debrief shows the shared summary while each learner keeps their own evidence.

Additional product capabilities

Optional voice uses local Whisper for transcription and configurable speech playback.
An optional Deep Focus browser extension supports focused study sessions.

4. AI model strategy: combining the two categories

The challenge asks for a deliberate split between compact/fast and larger/capable models. The shared LLM client maps task aliases to compact/capable tiers in `utils/llm_client.py` and model routes in `apps/ai/litellm/config.yaml`. Tutor rules select the alias per turn. This LLM workflow uses prompting and retrieval rather than project-specific LLM fine-tuning; optional speech and focus components have separate model configurations.

Category	Tasks (alias)	Why this category
Compact gpt-oss-20b	coach nudges; course-ingestion classification/extraction; Visual Lab *selector* (pick one of four registered engines); short tutor clarifications (`nardouch_compact`)	Short, well-defined, high-frequency steps: intent/engine classification, formatting, brief rewrites. Low latency, cheap to repeat.
Capable gpt-oss-120b (search/reflection: llama-3.3-70b)	tutor explanations, maths/code and misconception turns (`nardouch_capable`); Visual Lab *planner*; evaluator; peer facilitator proposal; course review	Multi-source synthesis against retrieved passages, structured planning and polished pedagogy where quality matters and a few extra seconds are acceptable.
No model	hint ladder, source-support lookup, confusion cues, common facilitation cases, grading of quizzes, Visual Lab numerics	Trusted code applies fixed rules and computes results without an LLM call. This avoids inference spend; it still needs tests and ordinary compute resources.

Routing rule (tutor). Only a small allow-list of exact clarification phrases in explain mode goes to the compact model. Message length alone never downgrades maths, code, misconceptions or follow-ups to the compact tier, to avoid quality loss on hard questions.
Declared external service. Development and the live probe used Groq-hosted open-weight models as stand-ins for the compact and capable categories. Aliases and `NARDOUCH_*_MODEL` overrides allow route changes. Organiser integration still requires a compatible provider adapter, credentials, matching approvals and response-identity checks; compatibility has not been tested.

5. Retrieval-augmented generation (RAG) and output control

- **Ingest:** native text or bounded local OCR → structure (chapters/lessons) → 200/50-word chunks → local embeddings → revision-pinned FAISS index and source manifest. Embeddings run locally: no learner text leaves the service for retrieval.
- **Retrieve:** the API resolves the learner's current authorisation and passes course, revision and manifest identity; the AI service returns only bounded excerpts from that pin.
- **Generate:** the prompt separates system instructions from learner text and passages; the model must return JSON (`displayText, sourceIds, followups`, plus optional speech fields) and say explicitly when course evidence is missing.
- **Verify:** the API checks response shape, pin and citation markers against the authorised evidence; Nardouch rejects foreign citations. The separate extractive tutor path can replace invalid citations with an authorised source excerpt. Provenance checks do not verify semantic entailment or guarantee a citation on every claim.

6. Personalisation loop

The six-question self-report probe selects a private, provisional wording cue for deterministic activity hints. Wrong attempts and hint escalation generate tentative confusion signals. Separately, an ordered adaptation policy uses explicit preferences, available lesson modes and eligible assessment evidence. The inspected Nardouch turn payload includes question, history, objective, mode and language; it does not automatically receive the probe profile. Preferences and confusion signals do not award mastery or change grades.

7. Efficient use and limited-capacity behaviour

The application is designed for shared inference capacity and whole-response delivery. The recorded live probe used hosted Groq inference; it does not establish CPU-only model execution or performance on the organiser platform.

Spending fewer, better requests

- **Budget ledger.** Each shared-client LLM call needs an unexpired operator approval bound to the task, the *exact* model, a request allowance and optional token/cost thresholds. The API reserves a request *before* dispatch (atomic conditional update) and records usage afterwards. Without usable approval or credentials, dispatch is blocked. The inspected Python path records token/cost use after completion; these thresholds are not hard per-call spend ceilings.
- **Bounded workflow calls.** The in-process router disables automatic retries and cross-model fallback, despite legacy YAML settings. A tutor turn dispatches once. Visual Lab generation uses a compact selector and capable planner, with at most one planner repair: up to three calls, each separately reserved.
- **Reuse.** The shared client offers opt-in Redis caching keyed by task, model, prompts and selected settings, with expiry. It applies only where callers enable it; not all planning paths do. A hit still consumes a request reservation. Private tutor dialogue does not opt in. Saved Visual Labs reopen without a new planning call.
- **One well-built request.** The tutor sends one request with retrieved passages and history instead of a chain of small calls; deterministic rules absorb the frequent cases (hints, common facilitator situations, quiz feedback).
- **Bounded concurrency.** The tutor limits in-flight turns per instance and answers “busy, retry shortly” rather than queuing unbounded work.

When capacity is limited or unavailable

Situation	Behaviour the user sees
Normal latency (several seconds)	Staged status and loading states while the whole response is awaited; long jobs (ingestion, review) show progress events from the worker queue; the input is kept if a request fails.
Approval exhausted/expired or provider unavailable	Explicit `UNAVAILABLE` state with a clear message; the tutor preserves the learner's question for retry; Visual Lab offers a labelled built-in example; the facilitator returns labelled local peer-first guidance.
Capable model slow or down	Deterministic paths (hints, quiz grading, rules-based facilitation, lab numerics) keep working. The shared LLM result contract defines execution modes (`REAL_APPROVED`, `MOCK_TEST`, `DETERMINISTIC_FALLBACK`, `UNAVAILABLE`); relevant tutor and facilitator states are labelled in the interface.
Background worker failure	Retryable vs terminal classification; long-running work uses asynchronous progress reporting.

Honest limitation. Automatic *fallback from the capable to the compact model* is not enabled: our approvals are bound to an exact model, so a silent switch would break the budget guarantee. Recovery is therefore a visible retry plus deterministic guidance. Pre-authorising a compact fallback approval for each capable task is the planned next step. A real quota-exhaustion response from the organiser platform has not been observed.

Recorded latency (one synthetic Visual Lab workflow)

Call	Model / category	Latency	Tokens	Mode
Visual Lab selector	gpt-oss-20b · compact	4.951 s	936	REAL_APPROVED
Visual Lab planner	gpt-oss-120b · capable	2.077 s	1,747	REAL_APPROVED
Selector after approval expiry	—	0 ms in wrapper; no dispatch	not recorded	UNAVAILABLE

Single observations, not a benchmark or end-to-end timing; raw data in `docs/submission/evidence/LIVE-INFERENCE-EVALUATION.json`.

8. Peer collaboration

- **Room commands** carry an expected sequence and a stable command ID: stale actions are rejected and replays are idempotent. Peer chat has its own durable, sequence-numbered journal, so a missed WebSocket broadcast is repaired by fetching ordered history after reconnect.
- **Facilitation proposes, the room decides.** Rules first; an unmatched, eligible request passes bounded room context to the capable model for a structured proposal. NestJS validates shape, recipient and current room sequence before delivering it, and records whether it came from the model or from local rules.
- **Group vs private.** Learners discuss a shared activity, then submit private checks; the debrief separates the shared summary from each learner's own outcome. Another learner's score and evidence are never returned to a peer.

9. Privacy, ethics and accessibility

Safeguard	Implementation
Data minimisation	Seeded demo accounts and course content are synthetic. Extraction benchmarks also reference a private course PDF that is absent from the repository. Nardouch voice input is transient and transcribed locally; corrected text can become a stored conversation message. Optional focus/gaze components require a separate privacy review.
What reaches the model	Nardouch sends bounded excerpts, question, objective and short history to the configured LLM. Facilitator prompts can include participant names and recent dialogue. Embeddings are local. Optional ElevenLabs playback sends answer text to a separate provider with separate metering. No automatic PII detector is implemented.
Access control & isolation	The inspected course/room paths re-check resource grants and scope private attempts to their owner. Environment templates and redaction helpers support secret handling. This review does not certify all routes, logs, deployment settings or repository history as free of secrets.
Fairness & role	The inspected adaptation policy uses explicit preferences and eligible evidence; this is a design restriction, not a measured fairness result. The AI supports learning: instructors review and publish content, and AI-extracted material is labelled as such. Labs never count as mastery; learning-style output is provisional.
Accessibility	Multilingual text policy (§2), reading modes (including Read/Guided/Explore/Practice and Deep Dive when supported), semantic landmarks and skip link, keyboard-reachable controls, light/dark themes, responsive layout and sourced-text fallback for formulas. No formal accessibility audit is claimed.

10. Testing strategy and results

Check (historical report / repository documentation)	Result
TypeScript typecheck (API + web)	Reported pass
Production build (NestJS + Next.js)	Reported pass
Lint (API + web)	Reported: 0 errors, 3 warnings
API unit/service tests (Jest)	Reported: 1,351 passed, 101 suites
Web component tests (Vitest)	Attached PDF: 282 passed; current HTML/README: 286; HTML lists 42 files.
AI service (pytest)	Attached PDF: 1,292 passed; current HTML: 1,294 passed, 54 skipped, 15 failed. Failures are attributed to a missing private PDF; the cause requires raw logs and a rerun.
Fresh-database setup	Reported: migration, demo seed, login, course reader, lab and room browser checks
Browser e2e (Playwright, recorded 9 Nov)	Reported: 9 passed (auth, password reset, workspace, two-user join)
Integration (PostgreSQL, recorded 9 Nov)	Reported: 20 room/facilitator, 17 membership/enrolment, 12 public-policy checks

This review: source inspection and small deterministic checks only. Historical totals above were not rerun; no raw suite logs were identified. No live provider call was made. **Not yet done:** user testing with real learners or teachers (listed as a bonus criterion), native-speaker scoring of Darija output, and multi-instance realtime load testing.

11. Innovation

- **Peer-first facilitation** instead of an answer machine: the AI's default move is to ask, hint and invite a peer explanation, and it is forbidden from exposing private profiles or fabricating peers (covered by evaluation cases).
- **Trusted-component generation.** Models choose and parameterise one of four registered, validated engines (weighted graph, parameter model, 3-D geometry, function plot); trusted code computes all numbers. This avoids executing model-generated code; scenario relevance, engine correctness and deployment security still need validation.
- **Capacity-aware architecture** as a product feature: budget ledger, explicit execution modes, deterministic-first design and honest UI states, rather than an app that only works when the model does.
- **Institution and individual in one identity**, with separately enforced grants — the same engine serves a university course and a self-learner's own notes.

12. Business-model note

	Institutions (B2B2C)	Independent learners (B2C)
Target users	Universities, training centres and bootcamps in Tunisia and the wider MENA/francophone region; instructors and students.	Self-learners, exam-prep students, study communities.
Value proposition	Turn existing course material into guided practice and structured peer study in French/Arabic/Darija, with instructor control, private evidence and an insights dashboard — without replacing teachers.	A personal tutor and study-room companion that adapts to how they learn, free to start.
Adoption path	Free single-course pilot with one instructor and a voluntary cohort → measure activation, instructor prep time, cost per active learner → annual/cohort licence → add cohorts and departments.	Free tier with bounded AI use → upgrade to Pro for higher allowance and premium features → study-group referrals.
Revenue	Per-cohort or annual licence with an included AI allowance and support.	Monthly Pro subscription.

Sustainability beyond the hackathon

- **Unit economics are driven by inference.** The architecture already limits it: task-based model tiers, deterministic logic, opt-in response reuse and per-approval request budgets. Pricing will be set from pilot data (requests and tokens per active learner, cache hit rate, instructor effort), not guessed.
- **Deployment model:** the application stack and embeddings can be self-hosted; the recorded LLM calls used Groq. Optional speech adds local models or another provider. Hardware sizing and total operating cost have not been measured.
- **Existing foundations:** accounts, institution/enrolment model, subscription tiers and Stripe routes exist; institutional allowance integration, billing validation and pilot-based pricing remain to be completed.
- **Risks:** model quality and quota dependence (mitigated by deterministic paths and provider-agnostic aliases), instructor workload (mitigated by AI-drafted, human-reviewed content), data protection in institutions (mitigated by synthetic-data policy, scoped access and self-hosting).

No paying customer, partnership, price or demand validation is claimed; the model above is a plan to be validated by the pilot.

13. Roadmap and reproducibility

- Next: capable→compact fallback approvals, collaborative quizzes, auto-generated revision notes, teacher-facing usage/cost view, strict token/cost reservation, a pinned full-suite rerun, real-learner pilot and Darija quality scoring.
- **Repository:** https://github.com/ASHEN-IX/wie-act-challenge — source, setup instructions (`README.md`), `.env.example` templates with no credentials, screenshots and this report. Run locally in mock mode with no keys, or add `GROQ_API_KEY` and an approval (`docs/llm-config.md`) for real inference.

Source anchors: tutor: `apps/ai/agents/nardouch/orchestrator.py`; budgets: `apps/api/modules/study/src/services/llmBudget.ts`; lab: `apps/ai/agents/visual_lab/planner.py`.